



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных Технологий

Кафедра Вычислительной техники

ОТЧЕТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ

по дисциплине

«Разработка программно-аппаратного обеспечения
информационных и автоматизированных систем»

Выполнил студент группы ИКМО-03-25

Фомичев Р.А.

Принял старший преподаватель

Боронников А.С.

Лабораторная работа
выполнена

«__»_____2025 г.

«Зачтено»

«__»_____2025 г.

Москва 2025

Задание

1. Создать файл отчёта по лабораторной работе с титульным листом, оглавлением, постановкой задачи и ходом выполнения работы.
2. Создать и настроить новый проект в САПР Vivado для ПЛИС семейства Artix7 XC7A100TCSG324-1.
3. Создать модуль верхнего уровня иерархии проекта «Фи__LR_TOP.v», где «Ф» — первая буква фамилии, «И» — первая буква имени. Если в группе у нескольких студентов совпадают «Ф» и «И», то им следует добавить не повторяющуюся вторую букву на своё усмотрение.
4. Создать модуль контроллера UART «Фи__UART.v».
5. Описать содержимое модуля контроллера UART «Фи__UART.v».
6. Провести верификацию описанного модуля контроллера UART «Фи__UART.v» с помощью временных диаграмм и встроенного симулятора. Необходимо рассмотреть следующие случаи: приём данных без ошибки, приём данных с ошибкой формата кадра, приём данных с ошибкой чётности, приём данных с ошибкой формата кадра и чётности данных, передача слова данных.
7. Создать модуль конечного автомата «Фи__FSM.v».
8. Описать содержимое конечного автомата «Фи__FSM.v».
9. Создать модуль дешифратора из ASCII в HEX «Фи__DC_ASCII_HEX.v».
10. Описать содержимое модуля дешифратора из ASCII в HEX «Фи__DC_ASCII_HEX.v».
11. Провести верификацию описанного модуля дешифратора из ASCII в HEX «Фи__DC_ASCII_HEX.v» с помощью временных диаграмм и встроенного симулятора. Рассмотреть случаи преобразования всех шестнадцатеричных символов в кодировке ASCII, включая символы в

нижнем регистре, а также случаи, когда символ в кодировке ASCII не соответствует шестнадцатеричному числу.

12. Создать модуль дешифратора из HEX в ASCII «Фи__DC_HEX_ASCII.v».
13. Описать содержимое модуля дешифратора из HEX в ASCII «Фи__DC_HEX_ASCII.v».
14. Провести верификацию описанного модуля дешифратора из HEX в ASCII «Фи__DC_HEX_ASCII.v» с помощью временных диаграмм и встроенного симулятора. Рассмотреть случаи преобразования всех шестнадцатеричных чисел в символы в кодировке ASCII.
15. Создать модуль ПЗУ «Фи__ROM.v».
16. Описать содержимое модуля ПЗУ «Фи__ROM.v» согласно выданному варианту.
17. Описать модуль верхнего уровня иерархии проекта, используя созданные модули, а также модули делителя частоты «Фи__DIVIDER.v» и фильтра дребезга кнопки «Фи__BTN_FLTR» (найти можно в СДО).
18. В файле пользовательских ограничений (*.xdc) настроить распределение сигналов по выводам корпуса ПЛИС.
19. Выполнить этапы маршрута проектирования для синтеза, трансляции, реализации в кристалл ПЛИС и генерации файла конфигурации. В случае успешного завершения вернуться к оформлению отчёта. Иначе — найти и устранить ошибки.
20. Загрузить файл конфигурации *.bit в ПЛИС. В случае успешного конфигурирования показать преподавателю и вставить в отчёт результаты работы. Иначе — найти и устранить ошибки.

21.В заключении привести данные о числе задействованных ресурсов ПЛИС.

Вариант:

Фомичев Роман Алексеевич	
Количество триггеров в синхронизаторе:	3
Скорость передачи	19200
Проверка четности	Even
Количество стоповых битов	1
RATIO	16
Сообщение результата выполнения операции	«res- »
Сообщение ошибки данных не соответствующих формату	«erRor:fOrmaT daTA»
Сообщение ошибки четности данных	«ERrOr:PARiTY uaRt»
Сообщение ошибки формата кадра	«ERRoR- format DAtA UaRt»
Сообщение ошибки формата кадра и четности данных	«eRR-Uart»
Тип операции	вычитание
Разрядность принимаемых данных (в битах)	20
Разрядность результата (в битах)	112

Рисунок 1 — Данные варианта

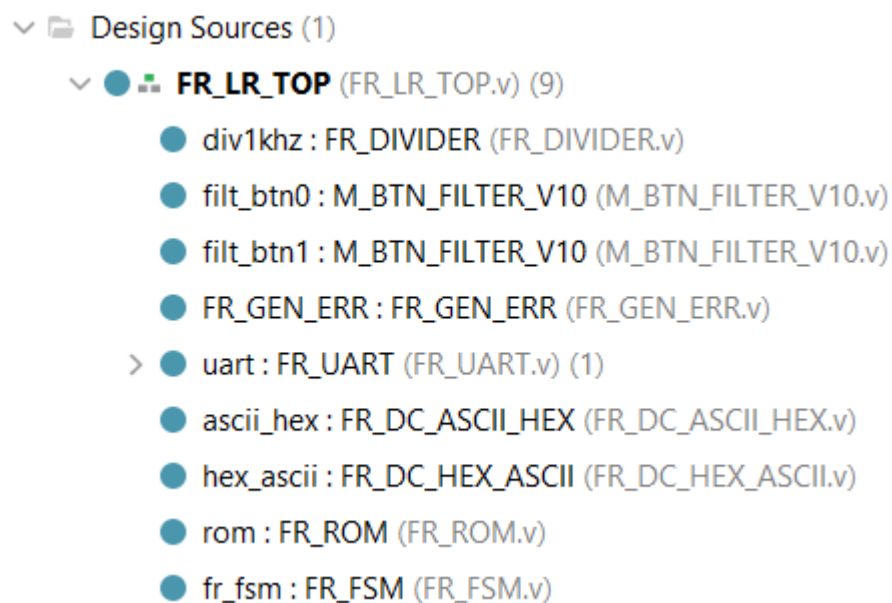


Рисунок 2 — Иерархия проекта

Листинг 1 — Модуль FR_LR_TOP

```

module FR_LR_TOP(
    input CLK,

```

```

        input SYS_NRST,
        input BTN_0,
        input BTN_1,
        input UART_RXD,
        output UART_TXD
    );

    wire RST;
    wire CE_1KHZ;

    wire GEN_FRT_ERR;
    wire GEN_PAR_ERR;

    wire [3:0] HEX_DATA;
    wire [7:0] ASCII_DATA;
    wire HEX_FLG;

    wire [3:0] DC_HEX_DATA;
    wire [7:0] DC_ASCII_DATA;

    wire [6:0] ADDR;
    wire [7:0] DATA;

    wire RX_DATA_EN;

    wire [9:0] RX_DATA;
    //wire [9:0] DATA_FSM;
    wire [9:0] RX_DATA_ERR;
    wire [7:0] TX_DATA;
    wire RX_RDY;
    wire [7:0] TX_RDY;

    reg [2:0] RST_SYNC = 3'b110;

    //Sync RST
    //-----
    always @(posedge CLK, negedge SYS_NRST) begin

```

```

        if (~SYS_NRST)
            RST_SYNC <= 3'b001;
        else
            RST_SYNC <= {RST_SYNC[1:0], 1'b0};
    end

assign RST = RST_SYNC[2];

//-----
//Divider 1kHz
FR_DIVIDER #(
    .DIVIDER(100000)
) div1khz(
    .CLK(CLK),
    .RST(RST),
    .CEO(CE_1kHz)
);
//-----
//-----
//Filter BTN-0
M_BTN_FILTER_V10 filt_btn0(
    .CLK(CLK),                // System Clock
    .CE(CE_1kHz),             // CE (1-2kHz - optimal)
    .BTN_IN(BTN_0),           // Asynch. Input From Button
    .RST(RST),                // Asynch. Reset
    .BTN_OUT(GEN_FRT_ERR)     // Filtered Output
//    .BTN_CEO()               // Clock Enable Pulse When L-H Transition
);
//-----
//Filter BTN-1
M_BTN_FILTER_V10 filt_btn1(
    .CLK(CLK),                // System Clock
    .CE(CE_1kHz),             // CE (1-2kHz - optimal)
    .BTN_IN(BTN_1),           // Asynch. Input From Button
    .RST(RST),                // Asynch. Reset
    .BTN_OUT(GEN_PAR_ERR)     // Filtered Output
//    .BTN_CEO()               // Clock Enable Pulse When L-H Transition
);

```

```

//-----
// CL GenErr
FR_GEN_ERR (
    .RX_DATA_T (RX_DATA) ,
    .GEN_FRT_ERR (GEN_FRT_ERR) ,
    .GEN_PAR_ERR (GEN_PAR_ERR) ,
    .DATA (RX_DATA_ERR)
);

// Контроллер UART
// -----
FR_UART uart (
    .CLK (CLK) ,
    .RST (RST) ,
    .RXD (UART_RXD) ,
    .TXD (UART_TXD) ,
    .RX_DATA_EN (RX_DATA_EN) ,
    .RX_DATA_T (RX_DATA) ,
    .TX_RDY_T (TX_RDY) ,
    .TX_DATA_R (TX_DATA) ,
    .TX_RDY_R (RX_RDY)
);

FR_DC_ASCII_HEX ascii_hex (
    .ASCII (ASCII_DATA) ,
    .HEX (DC_HEX_DATA) ,
    .HEX_FLG (HEX_FLG)
);

FR_DC_HEX_ASCII hex_ascii (
    .HEX (HEX_DATA) ,
    .ASCII (DC_ASCII_DATA)
);

FR_ROM rom (
    .ADDR (ADDR) ,
    .DATA (DATA)

```

```

);

FR_FSM fr_fsm(
    .CLK(CLK),
    .RST(RST),
    .RX_DATA_EN(RX_DATA_EN),
    .RX_DATA_R(RX_DATA_ERR),
    .TX_RDY_T(TX_RDY),
    .TX_DATA_T(TX_DATA),
    .TX_RDY_R(RX_RDY),
    .ASCII_DATA(ASCII_DATA),
    .HEX_FLG(HEX_FLG),
    .DC_HEX_DATA(DC_HEX_DATA),
    .HEX_DATA(HEX_DATA),
    .DC_ASCII_DATA(DC_ASCII_DATA),
    .ADDR(ADDR),
    .DATA(DATA)
);

endmodule

```

Листинг 2 — Модуль FR_DIVIDER

```

module FR_DIVIDER #(
    parameter DIVIDER = 100000
) (
    input CLK,
    input RST,
    output reg CEO
);

reg [$clog2(DIVIDER)-1:0] COUNTER;

always @(posedge CLK, posedge RST) begin
    if (RST) begin
        COUNTER = 0;
        CEO = 0;
    end
    else if (COUNTER == DIVIDER - 1) begin
        COUNTER = 0;
    end
end

```



```

        CEO = 1'b1;
    end
    else begin
        COUNTER = COUNTER + 1;
        CEO = 1'b0;
    end
end
end
endmodule

```

Листинг 3 — Модуль M_BTN_FILTER_V10

```

module M_BTN_FILTER_V10(
    input        CLK,                // System Clock
    input        CE,                 // CE (1-2kHz - optimal)
    input        BTN_IN,             // Asynch. Input From Button
    input        RST,               // Asynch. Reset
    output       BTN_OUT,            // Filtered Output
    output reg   BTN_CEO             // Clock Enable Pulse When L-H
    Transition
);

parameter [3:0] CNTR_WIDTH = 4;    // Internal Counter Width

// Internal signals declaration:
reg [CNTR_WIDTH - 1:0] FLTR_CNT;
reg BTN_D, BTN_S1, BTN_S2;
//-----

// Main Counter:
always @ (posedge CLK, posedge RST)
    if(RST) FLTR_CNT <= {CNTR_WIDTH{1'b0}};
    else
        if(!(BTN_S1 ^ BTN_S2))      // if BTN_S1 = BTN_S2
            FLTR_CNT <= {CNTR_WIDTH{1'b0}}; // Return to Zero
        else if(CE)                 // else if Clock Enable
            FLTR_CNT <= FLTR_CNT + 1; // Increment
//-----

// Input Synchronizer:

```

```

always @ (posedge CLK, posedge RST)
    if(RST)
        begin
            BTN_D   <= 1'b0;
            BTN_S1  <= 1'b0;
        end
    else
        begin
            BTN_D   <= BTN_IN;
            BTN_S1  <= BTN_D;
        end
//-----
// Output Register:
always @ (posedge CLK, posedge RST)
    if(RST) BTN_S2 <= 1'b0;
    else if(&(FLTR_CNT) & CE) BTN_S2 <= BTN_S1;
//-----
// Output Front Detector Clock Enable:
always @ (posedge CLK, posedge RST)
    if(RST) BTN_CEO <= 1'b0;
    else BTN_CEO <= &(FLTR_CNT) & CE & BTN_S1;
//-----
assign BTN_OUT = BTN_S2;
//-----
endmodule

```

Листинг 4 — Модуль FR_GEN_ERR

```

module FR_GEN_ERR(
    input  [9:0] RX_DATA_T,
    input  GEN_FRT_ERR,
    input  GEN_PAR_ERR,
    output [9:0] DATA
);

assign DATA[7:0] = RX_DATA_T[7:0];
assign DATA[8]   = RX_DATA_T[8] ^ GEN_PAR_ERR;

```

```

assign DATA[9]    = RX_DATA_T[9] ^ GEN_FRT_ERR;
endmodule

```

Листинг 5 — Модуль FR_UART

```

module FR_UART(
    input CLK,
    input RST,
    input RXD,
    output reg TXD,
    output reg RX_DATA_EN,
    output reg [9:0] RX_DATA_T,
    input TX_RDY_T,
    input [7:0] TX_DATA_R,
    output reg TX_RDY_R
);

reg [2:0] SYNC;
reg [2:0] RX_STATE;
reg [2:0] TX_STATE;
reg [2:0] RX_DATA_CT;
reg [2:0] TX_DATA_CT;
reg [3:0] RX_SAMP_CT;
reg [3:0] TX_SAMP_CT;
reg [7:0] TX_DATA;
reg TX_PAR_BIT_RG;
reg RXCT_R;
reg TXCT_R;

wire RXD_RG;
wire UART_CE;
wire RX_CE;
wire TX_CE;

//-----
localparam IDLE = 3'd0,
            RSTRB = 3'd1,
            RDT = 3'd2,

```

```

        RPARB = 3'd3,
        RSTB1 = 3'd4,
        RSTB2 = 3'd5,
        WEND = 3'd6;

//-----

localparam WCE = 3'd1,
        TSTRB = 3'd2,
        TDT = 3'd3,
        TPARB = 3'd4,
        TSTB1 = 3'd5,
        TSTB2 = 3'd6;

//-----

//Sync
always@(posedge CLK, posedge RST)
    if(RST)
        SYNC <= 3'b111;
    else
        SYNC <= {SYNC[1:0], RXD};

assign RXD_RG = SYNC[2];

//-----

//Делитель частоты
FR_DIVIDER #(
    .DIVIDER(5208)
)div1khz(
    .CLK(CLK),
    .RST(RST),
    .CEO(UART_CE)
);

//-----

//RX_FSM
always@(posedge CLK, posedge RST)
    if(RST)begin
        RX_STATE <= IDLE;

```

```

    RX_DATA_EN <= 1'b0;
    RX_DATA_T <= 10'h000;
    RX_DATA_CT <= {3{1'b0}};
    RXCT_R <= 1'b1;
end
else
    case (RX_STATE)
        IDLE: begin
            if (~RXD_RG) begin
                RX_DATA_EN <= 1'b0;
                RX_DATA_T[9] <= 1'b0;
                RXCT_R <= 1'b0;
                RX_STATE <= RSTRB;
            end
            else
                RX_DATA_EN <= 1'b0;
            end
        end

        RSTRB: begin
            if (RX_CE) begin
                if (RXD_RG) begin
                    RXCT_R <= 1'b1;
                    RX_STATE <= IDLE;
                end
                else
                    RX_STATE <= RDT;
                end
            end
        end

        RDT: begin
            if (RX_CE) begin
                if (RX_DATA_CT == 4'h7) RX_STATE <= RPARB;
                RX_DATA_T[7:0] <= {RXD_RG, RX_DATA_T[7:1]};
                RX_DATA_CT <= RX_DATA_CT + 1'b1;
            end
        end
    endcase
end

```

```

        end
    end

    RPARB: begin
        if(RX_CE) begin
            RX_DATA_T[8] <= RXD_RG ^ 1'b0; //МЕГА ВОПРОС
            RX_STATE <= RSTB1;
        end
    end

    RSTB1: begin
        if (RX_CE) begin
            RX_DATA_T[9] <= ~RXD_RG;
            RX_STATE <= RSTB2;
        end
    end

    RSTB2: begin
        if (RX_CE) begin
            if (RXD_RG) begin
                RX_DATA_EN <= 1'b1;
                RXCT_R <= 1'b1;
                RX_STATE <= IDLE;
            end
            else begin
                RX_DATA_T[9] <= 1'b1; // framing error: стоп-бит 2 = 0
                RX_STATE <= WEND;
            end
        end
    end

    WEND: begin
        if(RXD_RG) begin
            RX_DATA_EN <= 1'b1;
            RXCT_R <= 1'b1;
        end
    end
end

```

```

        RX_STATE <= IDLE;

    end

    end

    endcase

//-----
//TX-FSM

always@(posedge CLK,posedge RST) begin
    if(RST) begin
        TX_STATE <= IDLE;
        TX_DATA <= 8'h00;
        TX_PAR_BIT_RG <= 1'b0;
        TX_RDY_R <= 1'b1;
        TX_DATA_CT <= 3'b000;
        TXD <= 1'b1;
        TXCT_R <= 1'b1;
    end
    else case (TX_STATE)
        IDLE: if (TX_RDY_T) begin
            TX_DATA <= TX_DATA_R;
            TX_PAR_BIT_RG <= 0; //МЕГА ВОПРОС
            TX_RDY_R <= 1'b0;
            if (UART_CE) begin
                TXD <= 1'b0;
                TXCT_R <= 1'b0;
                TX_STATE <= TSTRB;
            end
            else TX_STATE <= WCE;
        end
        WCE: if (UART_CE) begin
            TXD <= 1'b0;
            TXCT_R <= 1'b0;
            TX_STATE <= TSTRB;
        end
        TSTRB: if (TX_CE) begin
            TXD <= TX_DATA[0];

```

```

        TX_DATA <= {1'b0, TX_DATA[7:1]};
        TX_STATE <= TDT;
    end

    TDT: if (TX_CE) begin
        TX_DATA <= {1'b0, TX_DATA[7:1]};
        TX_DATA_CT <= TX_DATA_CT + 1'b1;
        if (TX_DATA_CT == 3'd7) begin
            TXD <= TX_PAR_BIT_RG;
            TX_STATE <= TPARB;
        end
    end
    else begin
        TXD <= TX_DATA[0];
    end
end

end

TPARB: if (TX_CE) begin
    TXD <= 1'b1;
    TX_STATE <= TSTB1;
end

TSTB1: if (TX_CE) begin
    TXD <= 1'b1;
    TX_STATE <= TSTB2;
end

TSTB2: if (TX_CE) begin
    TX_RDY_R <= 1'b1;
    TXCT_R <= 1'b1;
    TX_STATE <= IDLE;
end

endcase
end

//-----
//RX-SAMP-CT
always@(posedge CLK, posedge RST)
    if (RST)
        RX_SAMP_CT <= 0;

```



```

    else if (RXCT_R)
        RX_SAMP_CT <= 0;
    else if (UART_CE)
        RX_SAMP_CT <= RX_SAMP_CT + 1'b1;

assign RX_CE = (RX_SAMP_CT == 4'd7) & UART_CE;
//-----
//TX-SAMP-CT
always@(posedge CLK, posedge RST)
    if(RST)
        TX_SAMP_CT <= 0;
    else if (TXCT_R)
        TX_SAMP_CT <= 0;
    else if (UART_CE)
        TX_SAMP_CT <= TX_SAMP_CT + 1'b1;

assign TX_CE = (TX_SAMP_CT == 4'd15) & UART_CE;
endmodule

```

Листинг 6 — Модуль FR_DC_ASCII_HEX

```

module FR_DC_ASCII_HEX(
    input [7:0] ASCII,
    output reg [3:0] HEX,
    output reg HEX_FLG
);

always @(*) begin
    case (ASCII)
        8'h30: begin HEX = 4'h0; HEX_FLG = 1'b1; end
        8'h31: begin HEX = 4'h1; HEX_FLG = 1'b1; end
        8'h32: begin HEX = 4'h2; HEX_FLG = 1'b1; end
        8'h33: begin HEX = 4'h3; HEX_FLG = 1'b1; end
        8'h34: begin HEX = 4'h4; HEX_FLG = 1'b1; end
        8'h35: begin HEX = 4'h5; HEX_FLG = 1'b1; end
        8'h36: begin HEX = 4'h6; HEX_FLG = 1'b1; end
        8'h37: begin HEX = 4'h7; HEX_FLG = 1'b1; end
    endcase
end

```

```

8'h38: begin HEX = 4'h8; HEX_FLG = 1'b1; end
8'h39: begin HEX = 4'h9; HEX_FLG = 1'b1; end
8'h41, 8'h61: begin HEX = 4'hA; HEX_FLG = 1'b1; end
8'h42, 8'h62: begin HEX = 4'hB; HEX_FLG = 1'b1; end
8'h43, 8'h63: begin HEX = 4'hC; HEX_FLG = 1'b1; end
8'h44, 8'h64: begin HEX = 4'hD; HEX_FLG = 1'b1; end
8'h45, 8'h65: begin HEX = 4'hE; HEX_FLG = 1'b1; end
8'h46, 8'h66: begin HEX = 4'hF; HEX_FLG = 1'b1; end
default: begin HEX = 4'h0; HEX_FLG = 1'b0; end

endcase

end

endmodule

```

Листинг 7 — Модуль FR_DC_HEX_ASCII

```

module FR_DC_HEX_ASCII(
    input [3:0] HEX,
    output reg [7:0] ASCII
);

always @(*) begin
    case (HEX)
        4'h0: ASCII = 8'h30;
        4'h1: ASCII = 8'h31;
        4'h2: ASCII = 8'h32;
        4'h3: ASCII = 8'h33;
        4'h4: ASCII = 8'h34;
        4'h5: ASCII = 8'h35;
        4'h6: ASCII = 8'h36;
        4'h7: ASCII = 8'h37;
        4'h8: ASCII = 8'h38;
        4'h9: ASCII = 8'h39;
        4'hA: ASCII = 8'h41;
        4'hB: ASCII = 8'h42;
        4'hC: ASCII = 8'h43;
        4'hD: ASCII = 8'h44;

```

```

        4'hE: ASCII = 8'h45;
        4'hF: ASCII = 8'h46;
    endcase
end

```

```

endmodule

```

Листинг 8 — Модуль FR_ROM

```

module FR_ROM(
    input  [6:0] ADDR,
    output [7:0] DATA
);

    reg [7:0] ROM [0:127];

    initial $readmemh("ROM0.mem", ROM);

    assign DATA = ROM[ADDR];

endmodule

```

Листинг 9 — Модуль FR_FSM

```

module FR_FSM(
    input CLK,
    input RST,
    input RX_DATA_EN,
    input [9:0] RX_DATA_R,
    output reg TX_RDY_T,
    output reg [7:0] TX_DATA_T,
    input TX_RDY_R,
    output [7:0] ASCII_DATA,
    input HEX_FLG,
    input [3:0] DC_HEX_DATA,
    output reg [3:0] HEX_DATA,
    input [7:0] DC_ASCII_DATA,
    input [7:0] DATA,

```

```

        output reg [6:0] ADDR
    );

    reg [3:0] STATE;
    reg [67:0] RES_REG;
    reg [4:0] RES_CT;
    reg [6:0] ERR_A0_MX;
    reg [6:0] ERR_A1_MX;
    reg [35:0] DATA_REG;
    reg [3:0] DATA_CT;
    reg [6:0] END_ADDR;
    reg RES_FLG;

    wire [6:0] RES_A0;
    wire [6:0] RES_A1;

    //-----
    localparam IDLE = 4'd0,
               RDT = 4'd1,
               RCR = 4'd2,
               RLF = 4'd3,
               TRES = 4'd4,
               TMEM = 4'd5,
               TDT = 4'd6,
               TCR = 4'd7,
               TLF = 4'd8;

    //-----
    assign ASCII_DATA = RX_DATA_R[7:0];

    //-----

    always@(*)
        case (RES_CT)
            0: HEX_DATA = RES_REG[67:64];
            1: HEX_DATA = RES_REG[63:60];
            2: HEX_DATA = RES_REG[59:56];

```

```

3: HEX_DATA = RES_REG[55:52];
4: HEX_DATA = RES_REG[51:48];
5: HEX_DATA = RES_REG[47:44];
6: HEX_DATA = RES_REG[43:40];
7: HEX_DATA = RES_REG[39:36];
8: HEX_DATA = RES_REG[35:32];
9: HEX_DATA = RES_REG[31:28];
10: HEX_DATA = RES_REG[27:24];
11: HEX_DATA = RES_REG[23:20];
12: HEX_DATA = RES_REG[19:16];
13: HEX_DATA = RES_REG[15:12];
14: HEX_DATA = RES_REG[11:8];
15: HEX_DATA = RES_REG[7:4];
16: HEX_DATA = RES_REG[3:0];

default: HEX_DATA = 4'b0;

endcase

//-----
assign RES_A0 = 6'd0;
assign RES_A1 = 6'd6;
//-----

always@(*)
    case(RX_DATA_R[9:8])
        2'b00: begin
            ERR_A0_MX = 7;
            ERR_A1_MX = 23;
        end
        2'b01: begin
            ERR_A0_MX = 24;
            ERR_A1_MX = 39;
        end
        2'b10: begin
            ERR_A0_MX = 40;
            ERR_A1_MX = 61;
        end
        2'b11: begin

```

```

        ERR_A0_MX = 62;

        ERR_A1_MX = 71;

    end

endcase

//-----

always @(posedge CLK, posedge RST) begin
    if (RST) begin
        TX_DATA_T <= 8'h00;

        TX_RDY_T <= 1'b0;

        DATA_CT <= {4{1'b0}}; //BOIPOC!

        RES_CT <= {5{1'b0}}; //BOIPOC!

        RES_REG <= {68{1'b1}}; //BOIPOC!

        DATA_REG <= {36{1'b0}}; //BOIPOC!

        ADDR <= {7{1'b0}};

        END_ADDR <= {7{1'b0}};

        RES_FLG <= 1'b0;

        STATE <= IDLE;

    end

    else case (STATE)

        IDLE: if (RX_DATA_EN) begin
            if (RX_DATA_R[9] | RX_DATA_R[8]) begin
                ADDR <= ERR_A0_MX;

                END_ADDR <= ERR_A1_MX;

                STATE <= TRES;

            end

            else if (HEX_FLG) begin
                ADDR <= RES_A0;

                END_ADDR <= RES_A1;

                DATA_REG <= {32'b0, DC_HEX_DATA};

                DATA_CT <= 4'd1;

                STATE <= RDT;

            end

            else begin
                ADDR <= 7;

```

```

        END_ADDR <= 23;

        STATE <= TRES;

    end

end

RDT: if (RX_DATA_EN) begin
    if (HEX_FLG) begin
        ADDR <= RES_A0;

        END_ADDR <= RES_A1;

        DATA_REG <= {DATA_REG[31:0], DC_HEX_DATA};

        DATA_CT <= DATA_CT + 1'b1;

        if (DATA_CT == 4'd8) begin
            DATA_CT <= {4{1'b0}};

            STATE <= RCR;

        end

    end

    else if (RX_DATA_R[9] | RX_DATA_R[8]) begin
        ADDR <= ERR_A0_MX;

        END_ADDR <= ERR_A1_MX;

        STATE <= TRES;

    end

    else begin
        ADDR <= 7;

        END_ADDR <= 23;

        STATE <= TRES;

    end

end

RCR: if (RX_DATA_EN) begin
    if (RX_DATA_R[7:0] == 8'h0D) begin
        STATE <= RLF;

    end

    else if (RX_DATA_R[9] | RX_DATA_R[8] | RX_DATA_R[7:0] != 8'h0D)
begin

        ADDR <= ERR_A0_MX;

        END_ADDR <= ERR_A1_MX;

        STATE <= TRES;

    end

end

```

```

end

RLF: if (RX_DATA_EN) begin
    if (RX_DATA_R[7:0] == 8'h0A) begin
        RES_REG <= RES_REG + DATA_REG;
        RES_FLG <= 1'b1;
        STATE <= TRES;
    end
    else if (RX_DATA_R[9] | RX_DATA_R[8] | RX_DATA_R[7:0] != 8'h0A)
begin
        ADDR <= ERR_A0_MX;
        END_ADDR <= ERR_A1_MX;
        STATE <= TRES;
    end
end

TRES: begin
    TX_DATA_T <= DATA;
    TX_RDY_T <= 1'b1;
    ADDR <= ADDR + 1'b1;
    STATE <= TMEM;
end

TMEM: if (TX_RDY_R) begin
    if (ADDR == END_ADDR + 1) begin
        if (RES_FLG) begin
            RES_FLG <= 1'b0;
            TX_DATA_T <= DC_ASCII_DATA;
            RES_CT <= RES_CT + 1'b1;
            STATE <= TDT;
        end
        else begin
            TX_DATA_T <= 8'h0D;
            STATE <= TCR;
        end
    end
    else begin
        TX_DATA_T <= DATA;
        ADDR <= ADDR + 1'b1;
    end
end

```



```

        end

    end

    TDT: if (TX_RDY_R) begin
        if (RES_CT == 5'd17) begin //ВОПРОС!
            TX_DATA_T <= 8'h0D;
            RES_CT <= {5{1'b0}}; //ВОПРОС!
            STATE <= TCR;
        end
        else begin
            TX_DATA_T <= DC_ASCII_DATA;
            RES_CT <= RES_CT + 1'b1;
        end
    end

    TCR: if (TX_RDY_R) begin
        TX_DATA_T <= 8'h0A;
        STATE <= TLF;
    end

    TLF: if (TX_RDY_R) begin
        TX_RDY_T <= 1'b0;
        STATE <= IDLE;
    end

endcase

end

endmodule

```

Листинг 10 — Файл ROM0.mem

```

52
65
53
55
6C
74
2D
45
52
52

```

4F
52
3A
66
4F
52
6D
41
74
20
64
41
74
41
45
72
72
2D
20
70
41
52
69
54
59
20
55
41
72
74
45
72
72
6F
52

2D
66
6F
72
6D
61
54
20
44
41
74
41
20
55
41
72
74
65
52
72
4F
72
3A
75
41
72
74